

Assignment Cover Sheet



Name(s) and Number(s) as per student card (s): Karnishwar R – 20086868, Anandha Krishnan S - 20055947

Student Programme: MSc Cyber Security

Lecturer Name: Swati Dongre

Module/Subject Title: Advanced Programming Techniques (B9CY108)

Assignment Title: B9CY108_CA_TWO_70%

No of Words: 3000

By submitting this assignment, I am/ we are confirming that:

- This assignment is all my/our own work;
- Any sources used have been referenced;
- I/we have followed the Generative AI instructions/ scale set out in the Assignment Brief;
- I/we have read the College rules regarding academic integrity in the [QAH Part B Section 3](#), and the [Generative AI Guidelines](#), and understand that penalties will be applied accordingly if work is found not to be my/our own.
- I/we understand that all work uploaded is submitted via Ou Original, whereby a text-matching report will show any similarities with other texts.

<u>S.No</u>	<u>Title</u>	<u>Page No.</u>
1	INTRODUCTION	3
2	Selection of Project Domain	3
3	Identification of System Requirements	4
4	Architectural Diagram	6
5	UML Diagrams	8
6	Data Structures and Algorithms Used	12
7	Tools and Technologies Used	13
8	Core Feature with Code Explanation	14
9	APIs Used	15
10	Errors Faced and How we fixed it	16
11	Testing Strategies	17
12	Future Improvements that can be done	17
13	Conclusion	18
14	Code Screenshots	18
15	Website Screenshots	22
16	References	26

17	GitHub and Demonstration Link	26	<u>Table</u> <u>Of</u>
----	----------------------------------	----	---------------------------

Contents:

1. INTRODUCTION:

- Secure Vault is a secure digital file storage and user activity management system.
- The main purpose of this system is to provide users with a secure file storage environment for storing personal files.
- The goal of this project is to allow users to upload, download, rename and delete files with strong security, user authentication system and complete transparency with activity logging.
- This system also includes an admin panel, only for users who are admin and allows admin access to everyone's files.
- We have also added a PDF export feature for user activity, so that the user can download their recent activities.
- There is also theme switching between light and dark modes, so that user can switch themes according to their requirement.
- The system was developed using **Streamlit** for the Front-End and python for the core logic in the system.
- **SQLite** is used as the database for our project for data storage.
- **ReportLab** is used for generating the PDF reports of user activity.
- Security features like password hashing, CAPTCHA-based login protection were implemented using python libraries and few other APIs.
- Custom CSS themes were implemented to provide a clean, user-friendly UI.
- Streamlit-extras stylable container was helpful in setting up the light and dark theme as well as setting up a background image.

2. Selection of Project Domain:

- The chosen project domain is secure file storage and activity management for our project.

- In today's digital world users need to frequently upload, view, store , manage and get files from online platforms.
- Some users also want to store files in a secure environment and easily access the files whenever they want.
- Eventhough there are many file storage apps, common issues such as insecure authentication, no activity monitoring, poor user interface and design makes the user feel unsafe on uploading their files in those sites.
- To overcome that issue, we have come up a system that has secure user authentication, user-based file access, user-friendly interface and activity logging to make it easy for the users.
- This project was chosen because it combines multiple important concepts such as authentication, authorization, file handling, database management, and user interface design. SecureVault addresses these problems by providing a single platform where files are securely stored and all actions are recorded for transparency.

3. Identification of System Requirements:

- We have built **SecureVault** with clear functional and non-functional requirements.
- Functional requirements represent what the system can do, whereas non-functional requirements represent how well the system does it.

3.1 Functional Requirements:

1) User Registration and Authentication:

- In this the user must be able to create account, and their passwords must be securely hashed.
- During this process the system should not allow weak passwords as well as the CAPTCHA validation must be used to prevent automated or robot logins.

2) File Management:

- In this the user must be able to easily upload, download, rename and delete files.

- The system should also maintain user specific directories and not display one user files to other users.

3) Activity Logging:

- With this functionality, users will be able to view their login, logout, file activities as well as activities related to ticket raising.

4) Admin Panel:

- With this feature, the user with admin control will have access to every users file for security purposes.

5) User Activity PDF:

- With this feature, users will be able to download their activity as a PDF file to Backtrack their activity if they encounter any problem.

6) UI and Themes:

- This functionality covers the CSS themes and other UI elements.
- A feature has been added to allow users to switch between light and dark modes according to their requirements.

7) Support:

- This allows users to report for bugs and any other problem they have encountered on the website.

3.2 Non-Functional Requirements:

1) Security:

- In this the system keeps users passwords safe by storing the plain text into a hashed version.

- This also ensures that only admin has access to other users files and user activity can accurately track what happened and when it happened with the correct timestamp.

2) Performance:

- The user should have a seamless experience with the website. The SQLite database helps needs to respond quickly so that the users don't feel any lag.

3) User-friendly:

- The website interface is built with Streamlit, which gives user a clean and confusion less UI experience.

4) Reliability:

- Eventhough there is no complex server setup, usage of SQLite database ensures that the user data is stored consistent and not mixed.

5) Maintainability:

- The code for this website is stored as modules.
- The core logic is separated into one module; UI logic is separated in another module.
- This reduces the complexity on the code and makes the system easier to update, fix and improve in later stages.

4. Architectural Diagram:

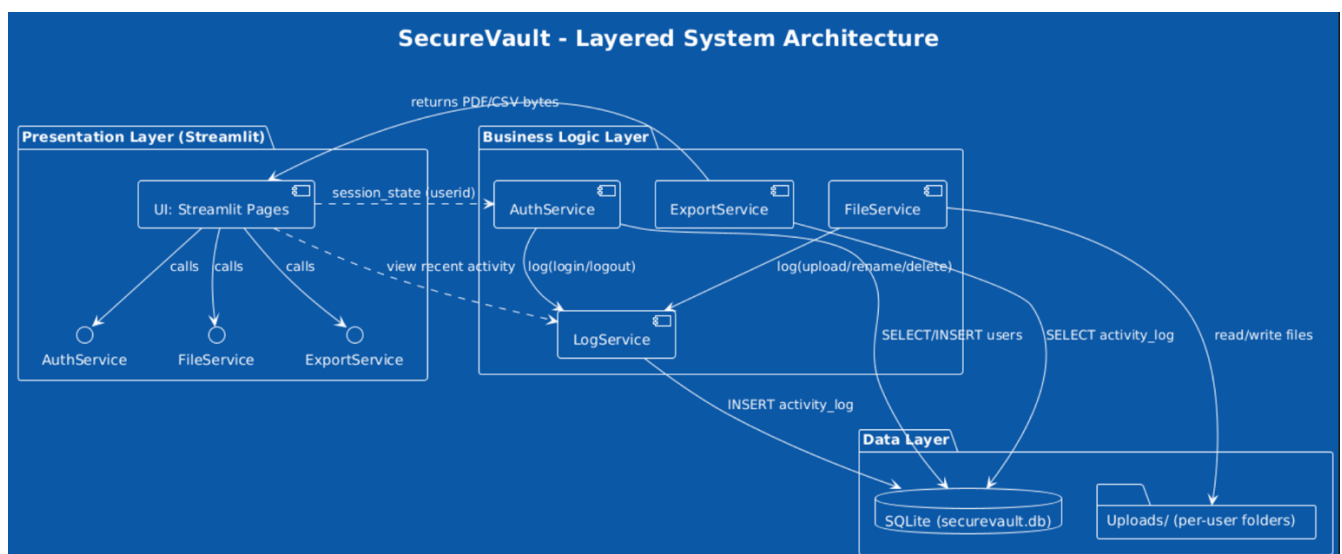


Image 1 – Architecture Diagram

- The system is split into **three layers**: Presentation, Business Logic, and Data.
- The **Presentation layer (Streamlit)** shows pages and sends requests.
- The **Business Logic layer** handles login, files, exports, and activity logs.
- The **Data layer** stores user data, activity logs, and uploaded files.
- Architectural Diagram is a high-level blueprint of how each component of the system work.
- **SecureVault** follows **Layered Architecture**.
- This system is divided into three main layers that are as follows:

4.1 Presentation Layer:

- This layer was implemented using **Streamlit**.
- This layer handles all the requirements related to UI.
- Like Handling user inputs, UI, buttons, forms and Navigation across pages.

4.2 Business Logic Layer:

- This layer is implemented using Python Functions and Libraries.
- This layer handles all the core tasks of the system.
- Like Authentication, Password hashing, CAPTCHA generation and validation, File Operations, Activity Logging and Activity Exporting.

4.3 Data Layer:

- This layer is implemented using
 - **SQLite database – Stores users information.**
 - **OS directory system – This is accessed using python libraries and used for storing uploaded files in the specified folder.**
 - **Base64 – This is an encoding mechanism and used for UI theme and Background images added on the website.**
- These implementations together help in:

- Independent modification of UI, logic and data.
- Better system management
- Good Modularity.

5) UML Diagrams:

5.1 Use Case Diagram:

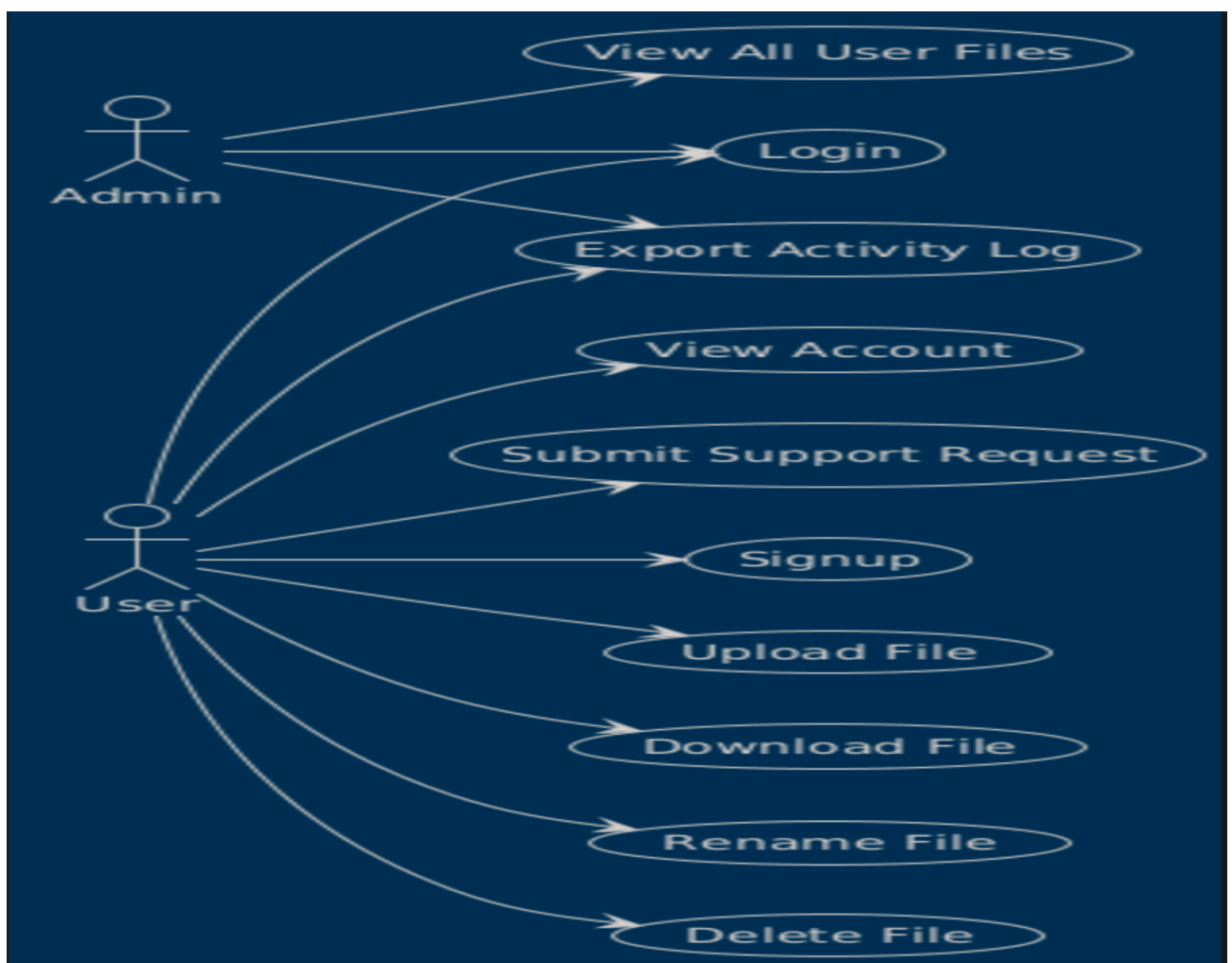


Image 2 – Use Case Diagram(UML)

- Image 2 shows a **file management system** with two roles: **User** and **Admin**.
- **User** can sign up, log in, upload files, download files, rename files, delete files, view their account, and submit support requests.

- **Admin** can log in and manage the system at a higher level.
- The **Admin** can view all user files and export the activity log.
- Both roles interact with the system, but **Admin has more control than User**.

5.2 Class Diagram:

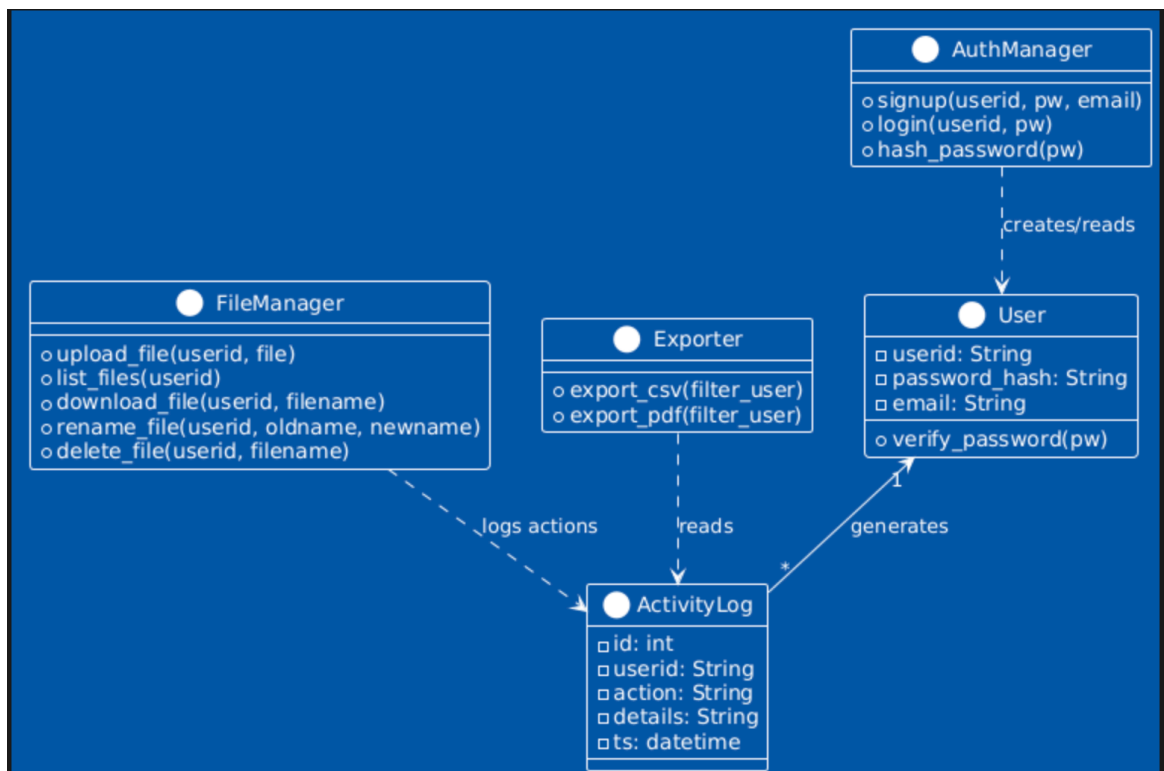


Image 3 – Class Diagram(UML)

- Image 3 shows the **main classes** used in the system and what they do.
- **AuthManager** handles signup, login, and password hashing for users.
- **User** stores user details and checks passwords.
- **FileManager** manages files: upload, list, download, rename, and delete.
- **ActivityLog** records user actions, and **Exporter** reads these logs to export data (CSV/PDF).

5.3 Activity Diagram(login):

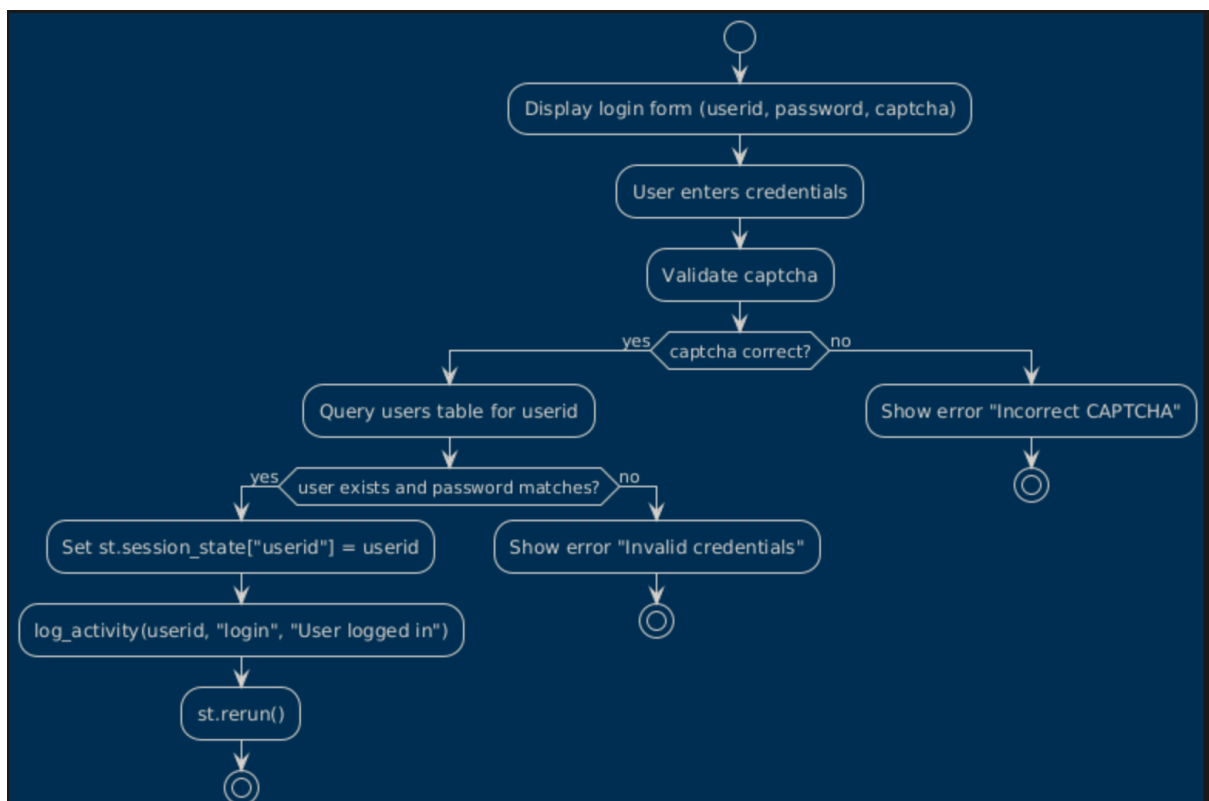


Image 4 – Activity Diagram(login)(UML)

- Image 4 shows a **login form** and the user enters their details.
- The system **checks the captcha** first.
- If the captcha is wrong, an **error message** is shown.
- If the captcha is correct, the system **checks the user ID and password**.
- If valid, the user is logged in and the **login action is saved**; otherwise, an error is shown.

5.4 Activity Diagram(file Upload):

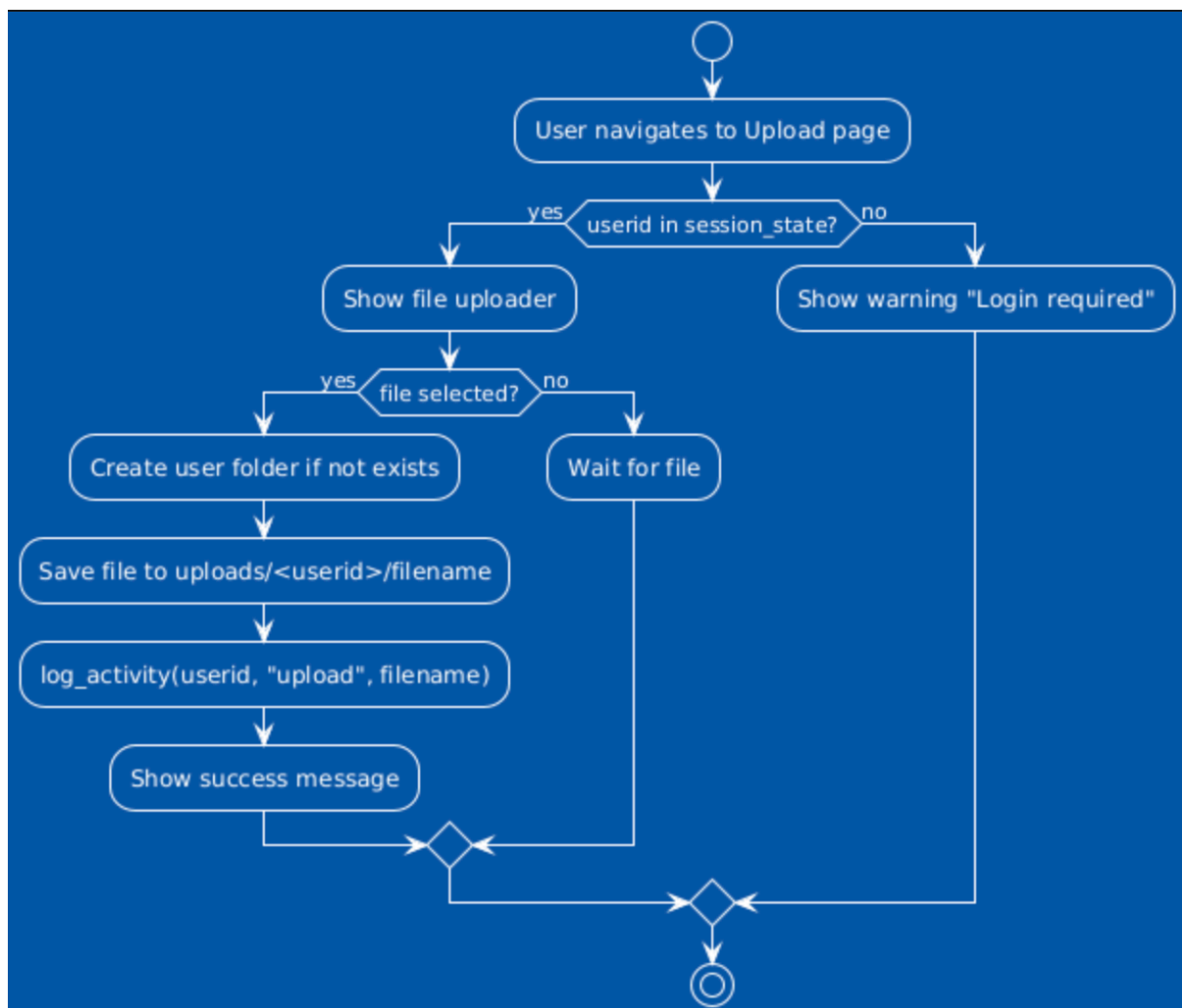


Image 5 – Activity Diagram(File Upload)(UML)

- The user opens the **Upload page**.
- The system checks if the user is **logged in**.
- If not logged in, it shows **“Login required”**.

- If logged in, the user selects a file and the system **saves it**.
- The upload is **logged** and a **success message** is shown.

6) Data Structures and Algorithms Used:

6.1 Data Structures Used:

- This system uses SQLite database tables to store user credentials and user activity in a Structured Format.
- A Directory-Based file structure is used, so that each user is assigned a separate directory, this assures user that their files are secure and no one has access to it except admin.
- Lists are used to store filenames. This is useful when a user needs to search for his file in the upload section. Lists filters the filenames.
- Strings are used to handle user input. That is after user enters password for login, it is immediately hashed and stored. Similarly for filenames and activity logging.
- Integers are used for generating CAPTCHA and after user enters the CAPTCHA integers also validate it.

6.2 Algorithms Used:

- In this system we have used **Password Hashing Algorithm SHA-256**. SHA-256 is a Cryptographic hashing algorithm used widely for password security.
- It ensures that passwords are converted in fixed length cipher text and cannot be reversed easily.
- A **Password Strength Evaluation Algorithm** is used.
- This uses a small loop to evaluate the strength of password given by the user.
- If the password does not meet the requirements, then it is recorded as Weak password and allows login only if the password is Medium or Strong.

- **CAPTCHA Validation Algorithm** is used to ensure that a user is trying to login and not a bot.
- This fixes automated attacks.
- **File Handling Algorithm** controls how files are uploaded, renamed, deleted and downloaded within a session.
- **Activity Logging Algorithm** saves each users activity along with the date and time into a database for easier tracking.
- **PDF Generation Algorithm** takes the activity log and formats it into a PDF file for a user to download.
- It also includes a backup option to download user activity as CSV if pdf fails.

7) Tools and Technologies Used:

7.1 Streamlit:

- Streamlit is used to build clean, interactive and responsive front-end interface. It mains a Good UI throughout a user session.

7.2 Python:

- Python was used for Handling core logic of the system. That includes authentication, file operations, security features, and activity logging efficiently.

7.3 SQLite:

- Database for storing user credentials, file data, and activity logs in a structured manner.

7.4 ReportLab:

- ReportLab was used to generate PDF files containing user and admin activity logs. This allows users to download their system activities.

7.5 PlantUML:

- PlantUML was used to design UML diagrams and system architecture diagrams. It helped in visually representing system workflows and component interactions.

- PlantUML allows generating these diagrams with the help of simple codes that represent each components activity.

7.6 Custom CSS:

- Custom CSS was implemented to improve the appearance of the application. It was used to create a modern layout and support light and dark theme styling.

7.7 Streamlit-extras stylable_container:

- This tool was used to create structured and visually consistent UI sections. It also played a key role in implementing theme switching and background image addition.

8) Core Feature with Code Explanation:

8.1 User Authentication:

- User authentication uses python and streamlit API.
- During signup “**hash_password()**” function applies SHA-256 algorithm to convert plain text to cipher text and store it in Database.
- Users also have a CAPTCHA verification, captcha is generated using pythons random module and stored in Streamlit’s **session_state**.
- Once user signup is done their details are stored in **session_state**.
- When user tries to login, the password is compared with the hash stored, if its matches then user data is retrieved from **st.session_state** and logs in the user.

8.2 File upload and Management:

- File upload is done using Streamlit **file_uploader()** API.
- When a user uploads a file, the system dynamically provides a user -specific directory for the logged in user with the user ID using the help of **os.makedirs()**.
- Uploaded files are saved using binary write operations for file integrity.
- **os.listdir()** – Lists all files belonging to the particular user.

- **os.rename()** – allows users to rename files.
- **os.remove()** – allows users to delete files.

8.3 User Activity Logs:

- Activity log is implemented through **log_activity()** function.
- This function accepts inputs such as user ID, action type, and other details like date and time, and then inserts the record into the **activity_log** table.
- Python's datetime module is used to generate the timestamp.
- This Activity Log function is called every time a major action occurs in the system like login, logout, uploading files, user submitting complaints .

8.4 PDF and CSV downloads for user activity:

- The activity export feature is implemented using ReportLibrary.
- The activity records are fetched from the database and written sequentially in the PDF.
- If the report library fails or any error occurs, then the system has a backup of downloading the activity as CSV files.

8.5 Admin Panel:

- So, admin access is verified simply by checking whether the logged in user ID has a new username called admin.
- This user ID admin is used as an identifier.

8.6 Theme Control:

- Theme Switching is enabled by **Streamlit session_state**
- So this function loads, the appropriate CSS file automatically based on the users theme selection and background images according to user selection.
- **Base64** is used for adding background images to the CSS and also the UI elements in the Secure Vault Website.

9. APIs Used:

- In this project, we have used Streamlit API for UI and User Session Handling
- SQLite API is used for database operations.
- Report Lab API is used for PDF generation.
- Python's standard library APIs like OS, HashLib, Random, Date, Time, and re were also used.

10. Errors Faced and How we fixed it:

10.1 Database Errors:

- Queries failed because tables did not exist or column names were mismatched.
- We created an initialization function (**init_activity_log()**) that ensures tables exist before use and standardized SQL queries.

10.2 File Handling Issues:

- Uploaded files were not saved because user folders didn't exist.
- We used **os.makedirs(user_folder, exist_ok=True)** to create directories automatically whenever needed.

10.3 Password Hash Mismatch:

- Logins failed even when the correct password was entered.
- Passwords were stored hashed, but comparison was done incorrectly in earlier versions.
- We Implemented a dedicated **hash_password()** function ensuring consistent hashing during signup and login.

10.4 Streamlit Rerun and State Issues:

- File rename/delete menus stayed open or reset unexpectedly due to Streamlit's rerun behavior.
- We used **st.session_state** to store selected file names, CAPTCHA values, and logged-in user state.

10.5 CSS Not Loading Properly:

- Theme switching didn't apply styles consistently.
- We loaded the CSS using a custom function (**load_css()**) and used Streamlit's **unsafe_allow_html=True** flag to inject styles correctly.

10.6 PDF Generation Errors:

- Long activity logs overflowed past the page bottom.
- Implemented a page-break algorithm

10.7 CAPTCHA Errors:

- Invalid inputs (letters instead of numbers) crashed the login function.
- Wrapped the CAPTCHA check in a try/except block to handle incorrect input safely.

11. Testing Strategies:

- Testing in the course of development We performed unit tests and integration tests. Each of the functions such as hashing passwords, CAPTCHA authentication, file operations, etc. were tested individually.
- We also did full end-to-end tests of the functionality of the login, uploading files, activity logging, and exporters to ensure that the UI, business logic and database are cooperating correctly.

12. Future Improvements that can be done:

1. File encryption can be put on stored files.
2. Email verification and password reset can be done.
3. Cloud Storage and Database can replace local storage and SQLite for better scalability.
4. Multiple admin levels can be created, and role-based access for each admin can be given.
5. Multi-factor authentication can be added.

13. Conclusion:

SecureVault demonstrates a secure file storage system. It has authentication, activity logs, administration, and data export. The project is realistic, well-modelled and security is considered. It also provides a good foundation of what can be improved and put into practice.

14) Code Screenshots:

```
def hash_password(password: str) -> str:  
    return hashlib.sha256(password.encode()).hexdigest() # uses hashing password for providing security
```

Image 6 – Password Hashing

```

if "captcha_a" not in st.session_state:
    st.session_state["captcha_a"] = random.randint(1, 9)
    st.session_state["captcha_b"] = random.randint(1, 9)

a = st.session_state["captcha_a"]
b = st.session_state["captcha_b"]
captcha_answer = st.text_input(f"Solve CAPTCHA: {a} + {b} = ?")

if st.button("Login"):
    try:
        if int(captcha_answer) != a + b:
            st.error("Incorrect CAPTCHA.")
            st.session_state["captcha_a"] = random.randint(1, 9)
            st.session_state["captcha_b"] = random.randint(1, 9)
            return
    except:
        st.error("Enter a valid number.")
        return # This provides a captcha for user before login

```

Image 7 – CAPTCHA Generation and Validation

```

uploaded_file = st.file_uploader("Choose a file")

if uploaded_file:
    user_folder = os.path.join(UPLOAD_FOLDER, st.session_state["userid"])
    os.makedirs(user_folder, exist_ok=True)
    file_path = os.path.join(user_folder, uploaded_file.name)

```

Image 8 – Folder creation for each user dynamically

```

if st.session_state.get("file_menu") == f:
    new_name = st.text_input("Rename to:", value=f, key=f"rn_{f}")
    c1, c2 = st.columns(2)
    if c1.button("Save name", key=f"sv_{f}"):
        new_path = os.path.join(user_folder, new_name)
        os.rename(file_path, new_path)
        log_activity(st.session_state["userid"], "rename", f"{f} → {new_name}")
        st.session_state["file_menu"] = None
        st.rerun()

```

Image 9 – File Rename

```

if c2.button("Delete", key=f"del_{f}"):
    os.remove(file_path)
    log_activity(st.session_state["userid"], "delete", f"Deleted {f}")
    st.session_state["file_menu"] = None
    st.rerun() # Logic for file viewing page and it also includes logic for file renaming as well as file deletion

```

Image 10 – File Delete

```
def init_activity_log():
    conn = get_db_connection()
    conn.execute(
        """
        CREATE TABLE IF NOT EXISTS activity_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            userid TEXT,
            action TEXT NOT NULL,
            details TEXT,
            ts DATETIME DEFAULT CURRENT_TIMESTAMP
        )
        """
    )
    conn.commit()
    conn.close() # user activity log table (structure)

def log_activity(userid, action, details=None):
    conn = get_db_connection()
    conn.execute(
        "INSERT INTO activity_log (userid, action, details, ts) VALUES (?, ?, ?, CURRENT_TIMESTAMP)",
        (userid, action, details),
    )
    conn.commit()
    conn.close() # insert user activity into the above table
```

Image 11 – Connecting to DB and user activity log Creation

```
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
import sqlite3

DB = "securevault.db"
OUTPUT = "activity_log.pdf"

def export_to_pdf():
    conn = sqlite3.connect(DB)
    cur = conn.cursor()

    cur.execute("SELECT userid, action, details, ts FROM activity_log ORDER BY ts DESC")
    rows = cur.fetchall()
    conn.close()

    c = canvas.Canvas(OUTPUT, pagesize=letter)
    width, height = letter

    x = 40
    y = height - 50

    c.setFont("Helvetica-Bold", 16)
    c.drawString(x, y, "SecureVault - User Activity Log")
    y -= 40

    c.setFont("Helvetica", 10)

    for row in rows:
        line = f"User: {row[0]} | Action: {row[1]} | Details: {row[2]} | Time: {row[3]}"
        c.drawString(x, y, line)
        y -= 15

        if y < 40:
            c.showPage()
            c.setFont("Helvetica", 10)
            y = height - 40

    c.save()
    print("PDF exported successfully ->", OUTPUT)

if __name__ == "__main__":
    export_to_pdf()
```

Image 12 – PDF Generator

```

from streamlit_extras.stylable_container import stylable_container

st.set_page_config(page_title="SecureVault", layout="centered", page_icon="🔒") # setting page details

if "theme" not in st.session_state:
    st.session_state["theme"] = "dark" # this command is for setting default theme as dark

with st.sidebar:
    st.markdown("### Appearance")
    theme_choice = st.radio("Theme", ["Dark", "Light"], horizontal=True)
    st.session_state["theme"] = "dark" if theme_choice == "Dark" else "light" # Mode changing Switch in Navigation Bar

def load_css(path):
    with open(path) as f:
        st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True) # load the css theme light or dark as per our selection

if st.session_state["theme"] == "dark":
    load_css("dark.css")
    set_bg("images/dark_bg.jpg")
else:
    load_css("light.css")
    set_bg("images/light_bg.jpg") # this is for the background image according to the theme we select

```

Image 13 – Light and Dark Mode

```

def set_bg(image_path):
    with open(image_path, "rb") as f:
        data = f.read()
        encoded = base64.b64encode(data).decode()
        st.markdown(
            f"""
            <style>
            [data-testid="stAppViewContainer"] {{
                background-image: url("data:image/jpg;base64,{encoded}");
                background-size: cover;
                background-position: center;
            }}
            </style>
            """,
            unsafe_allow_html=True,
        ) # this function to set background image

```

Image 14 – Background image

```

import sqlite3
import csv
import os

DB_NAME = "securevault.db"
OUTPUT_DIR = "db_exports"

os.makedirs(OUTPUT_DIR, exist_ok=True)

def export_table(table_name):
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()

    cur.execute(f"SELECT * FROM {table_name}")
    rows = cur.fetchall()

    col_names = [desc[0] for desc in cur.description]

    file_path = os.path.join(OUTPUT_DIR, f"{table_name}.csv")

    with open(file_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(col_names)
        writer.writerows(rows)

    conn.close()
    print(f"Exported {table_name} → {file_path}")

def main():
    export_table("users")
    export_table("activity_log")

if __name__ == "__main__":
    main()

```

Image 15 – Exporting Database as a CSV file

15) Website Screenshots:

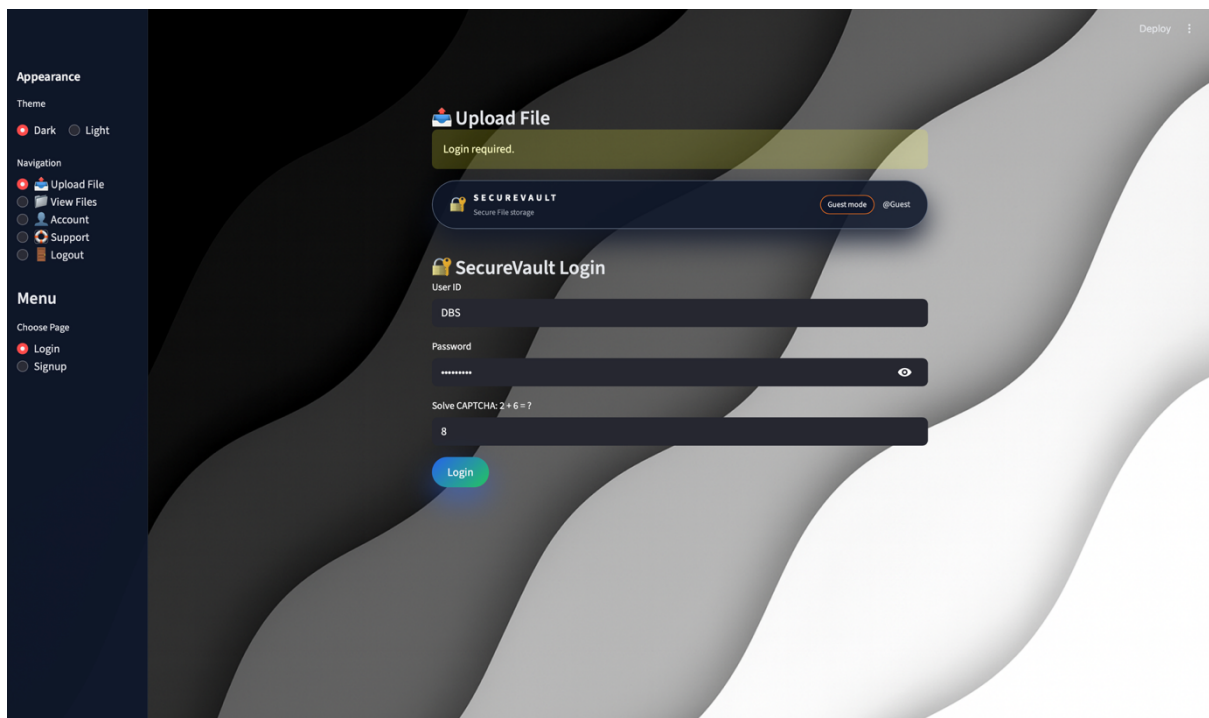


Image 16 – Login in Dark Mode



Image 17 – Login in Light Mode

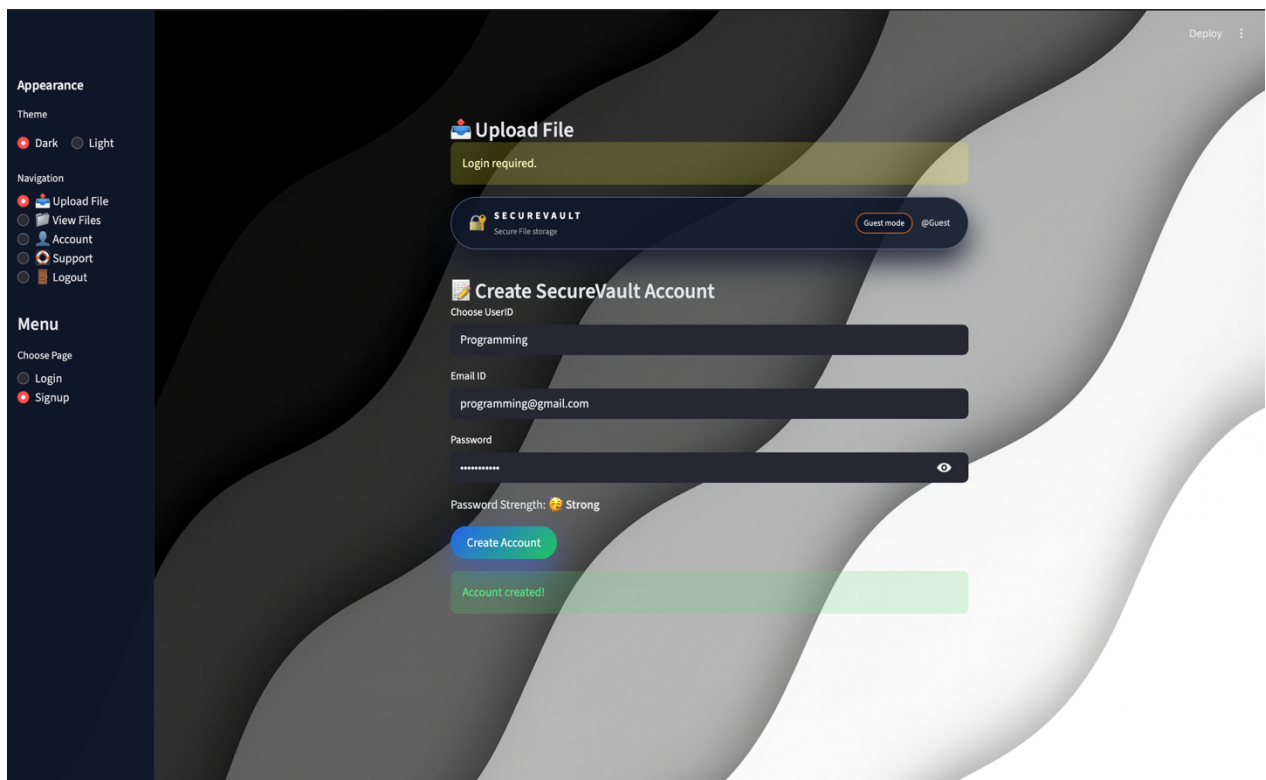


Image 18 – Signup with Password Strength

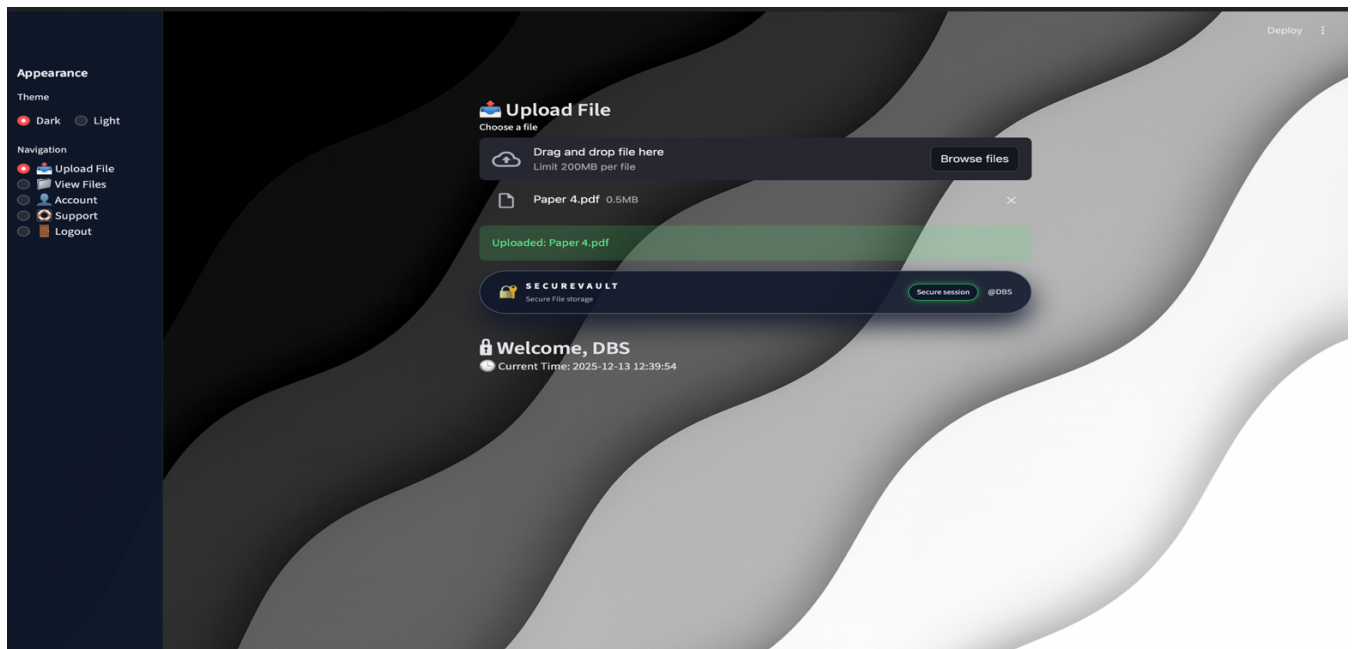


Image 19 – Welcome and File upload page

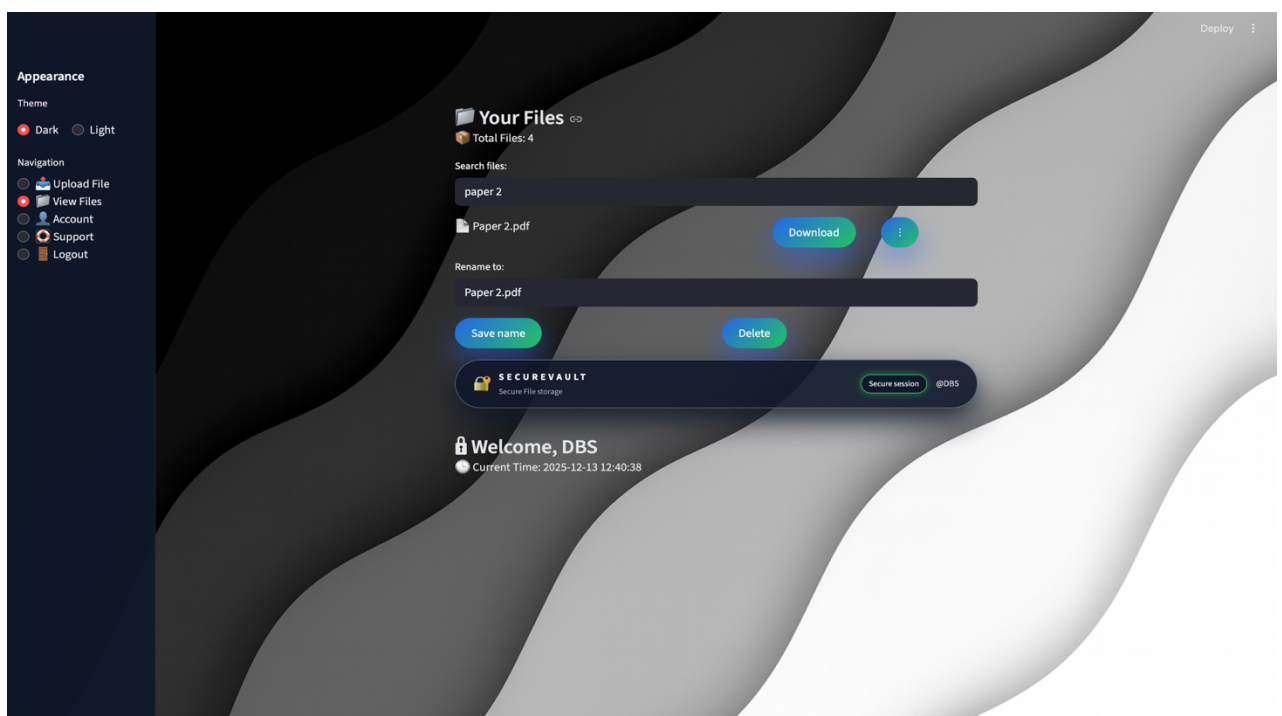


Image 20 – User file page with File search and file download, rename, delete.

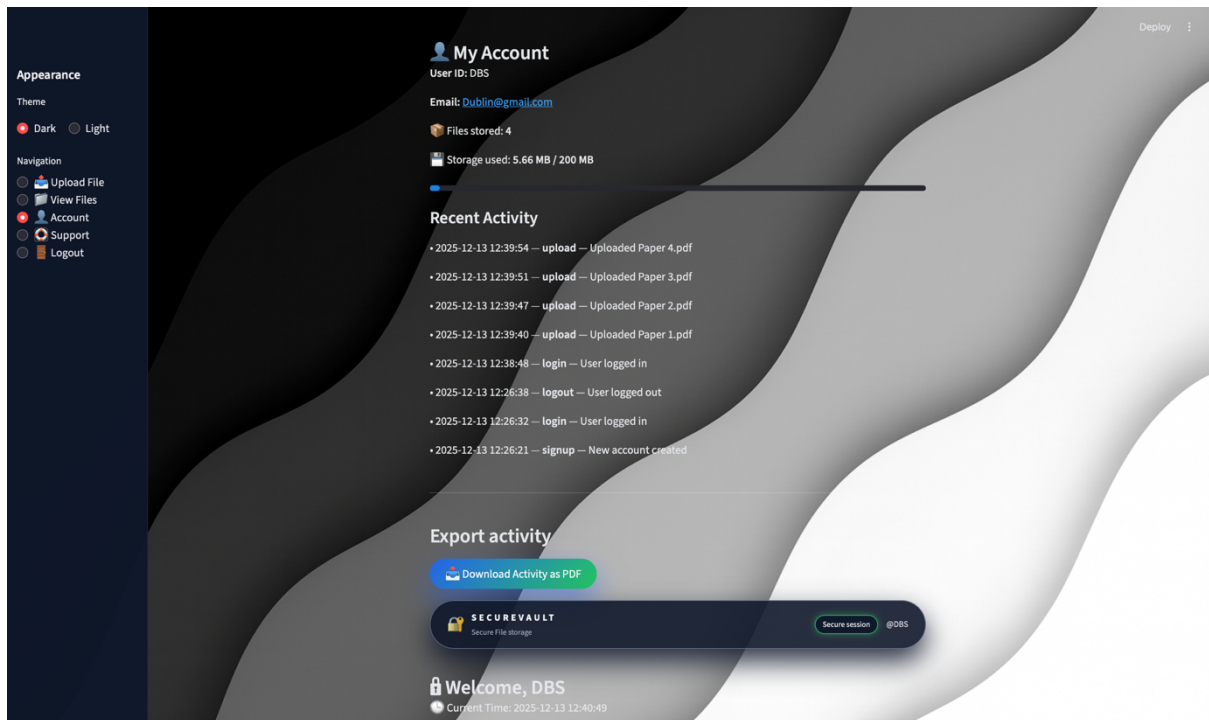


Image 21 – User activity logging with download as PDF and storage Bar

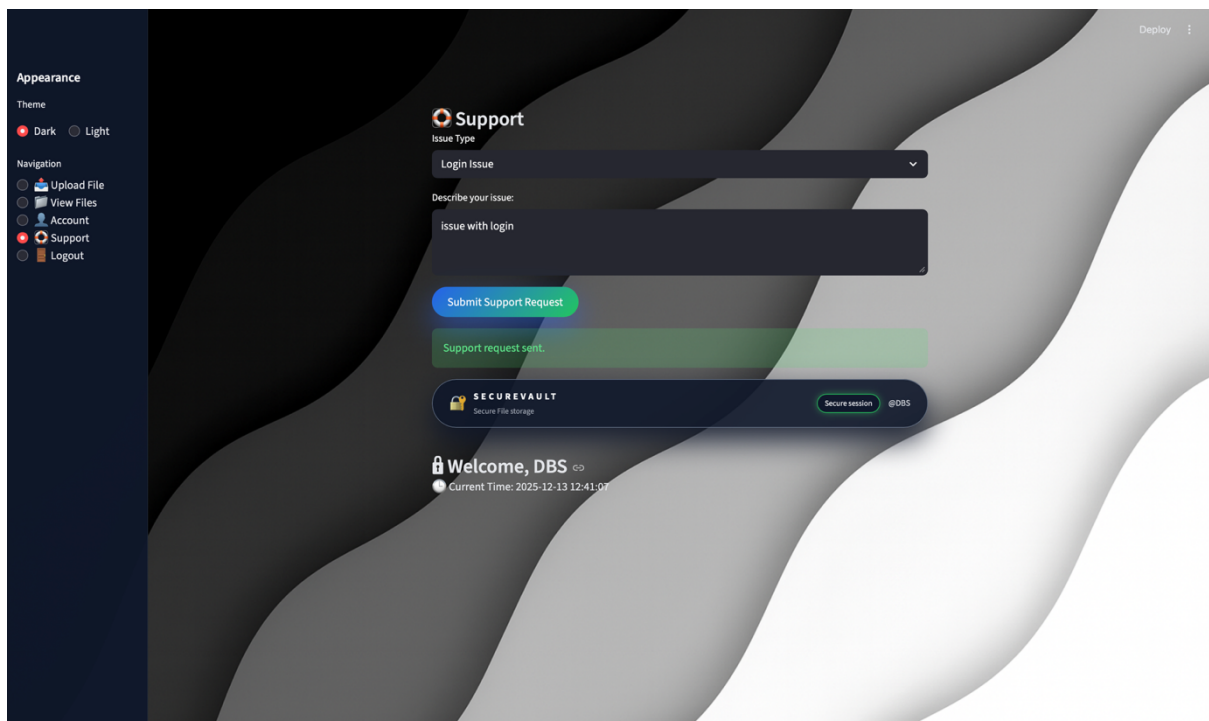


Image 22 – Support Page

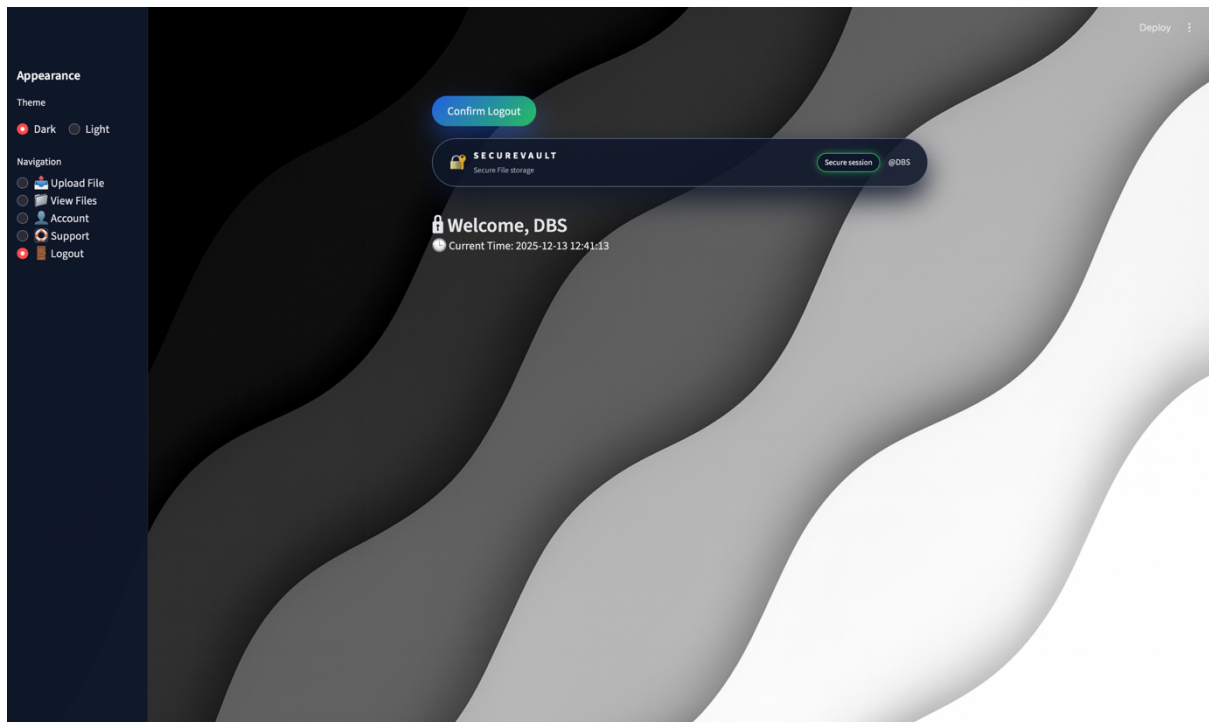


Image 23 – Logout Page

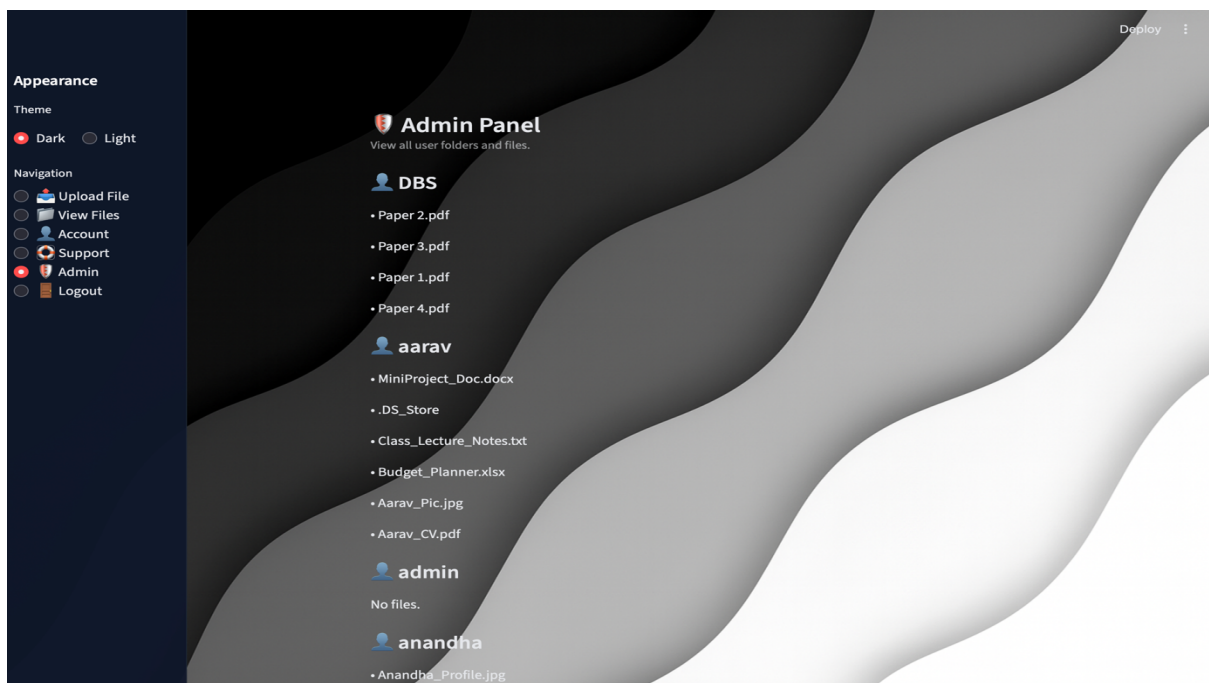


Image 24 – Admin page will all user files(only for admin)

16) References:

1. ReportLab User Guide - <https://docs.reportlab.com>
2. Streamlit Documentation - <https://docs.streamlit.io>
3. YouTube - [Streamlit + SQLite Tutorial 1: Add Data to Your First Web App with Python](#)
- (Playlist) by Turtle Code
4. YouTube - [Build a Python Website in 15 Minutes With Streamlit](#) – by Tech With Tim
5. YouTube - [File Uploader Widget - Web App with Python Streamlit Lesson 20](#) - by
Turtle Code

17) GitHub Link and Demonstration Link:

GitHub Link - <https://github.com/karnishwar1725/SecureVault.git>

Demonstration Link (Canva) - https://www.canva.com/design/DAG7bW7Scc0/281vdb5-SR43lQHbGmgQqA/watch?utm_content=DAG7bW7Scc0&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h1a3f8ffb1a